CS 645

Security and Privacy in Computer Systems

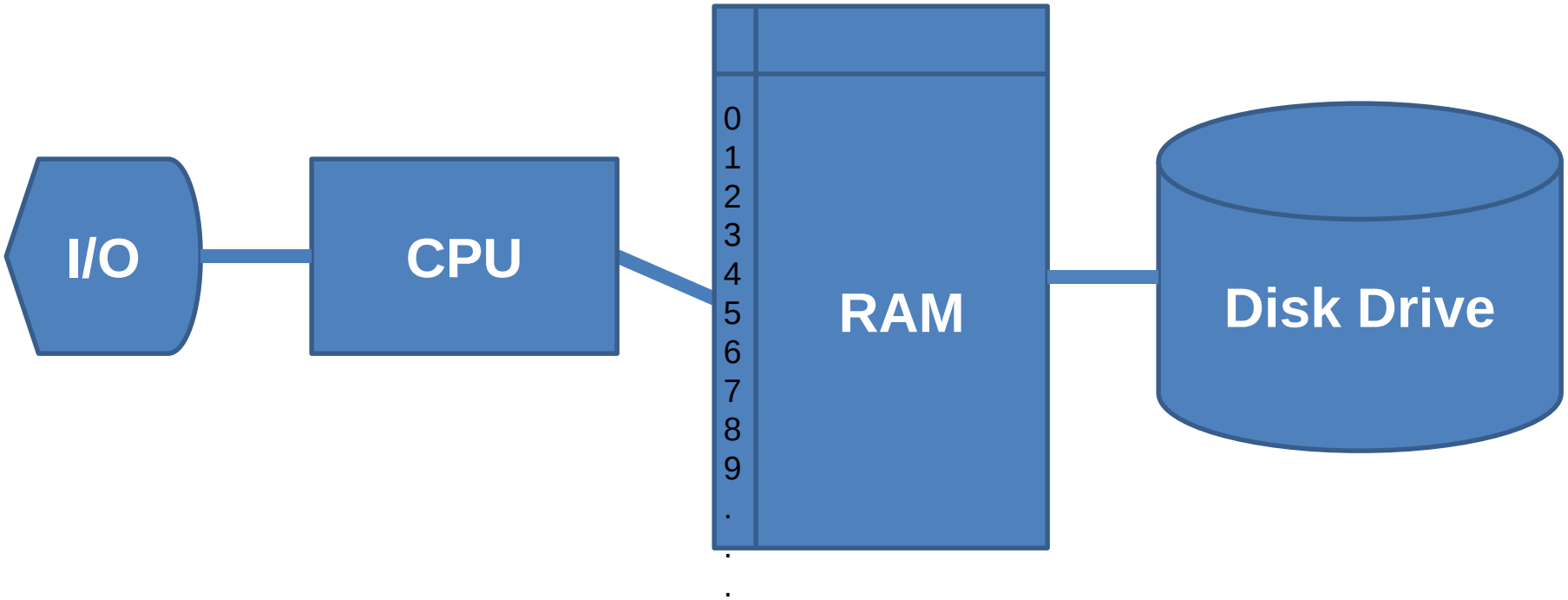
Lecture 4: OS and Filesystem Security

- Operating Systems Security
- Filesystem Security

Operating System Concepts
Operating System Security

A Computer Model

- An operating system has to deal with the fact that a computer is made up of a CPU, random access memory (RAM), input/output (I/O) devices, and long-term storage.



OS Concepts

- An **operating system (OS)** provides the interface between the users of a computer and that computer's hardware.
  - An operating system manages the ways applications access the resources in a computer, including its disk drives, CPU, main memory, input devices, output devices, and network interfaces.
  - An operating system manages multiple users (multiuser).
  - An operating system manages multiple programs (multitasking).

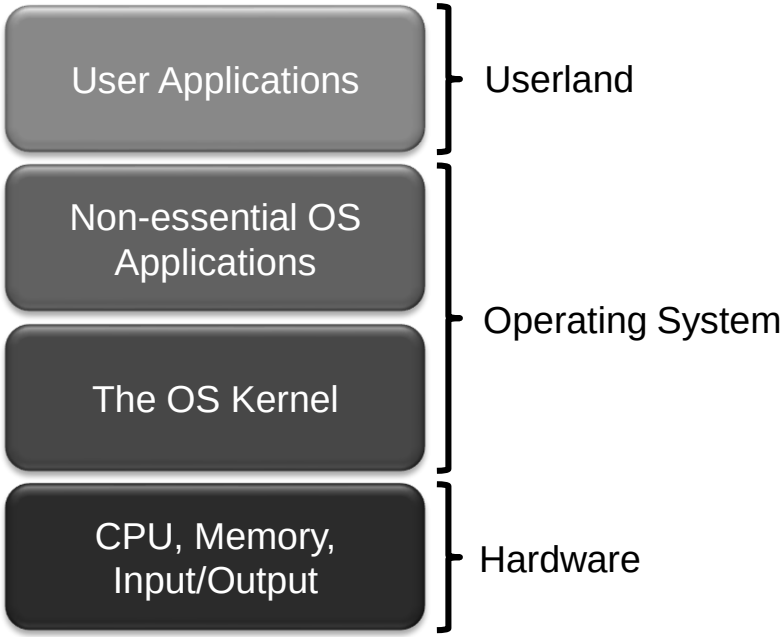
Multitasking

- Give each running program a “slice” of the CPU’s time.
- The CPU is running so fast that to any user it appears that the computer is running all the programs simultaneously.

Public domain image from http://commons.wikimedia.org/wiki/File:Chapters_meeting_2009_Liam_juggling.JPG

The Kernel

- **kernel** = core component of the OS
  - handles the management of low-level hardware resources, including memory, processors, and input/output (I/O) devices, such as a keyboard, mouse, or video display.
- **layer** metaphor
  - with the hardware components, such as the CPU, memory, and input/output devices being on the bottom, and users and applications being on the top.



Input/Output Devices

- **input/output devices** of a computer
  - keyboard, mouse, video display, and network card
  - other optional devices: scanner, Wi-Fi interface, video camera, USB ports, etc.
- Each such device is represented in an operating system using a **device driver,** which encapsulates the details of how interaction with that device should be done.
  - The **application programmer interface** (**API),** which the device drivers present to application programs, allows those programs to interact with those devices at a fairly high level
  - The operating system does the “heavy lifting” of performing the low-level interactions that make such devices actually work.

System Calls

- User applications don't communicate directly with low-level hardware components, and instead delegate such tasks to the OS kernel via **system calls (syscall)**.
- System calls are usually contained in a collection of programs, that is, a **library** such as the C library (libc)
  - they provide an interface that allows applications to use a predefined series of APIs that define the functions for communicating with the kernel.
  - Examples of system calls include those for performing file I/O (open, close, read, write) and running application programs (exec).

Processes

- A **process** is an instance of a program that is currently executing.
- The actual contents of all programs are initially stored in persistent storage, such as a hard drive.
- In order to be executed, a program must be loaded into random-access memory (RAM) and uniquely identified as a process.
- In this way, multiple copies of the same program can be run as different processes.
  - For example, we can have multiple copies of the Firefox browser open at the same time.

Users and the Process Tree

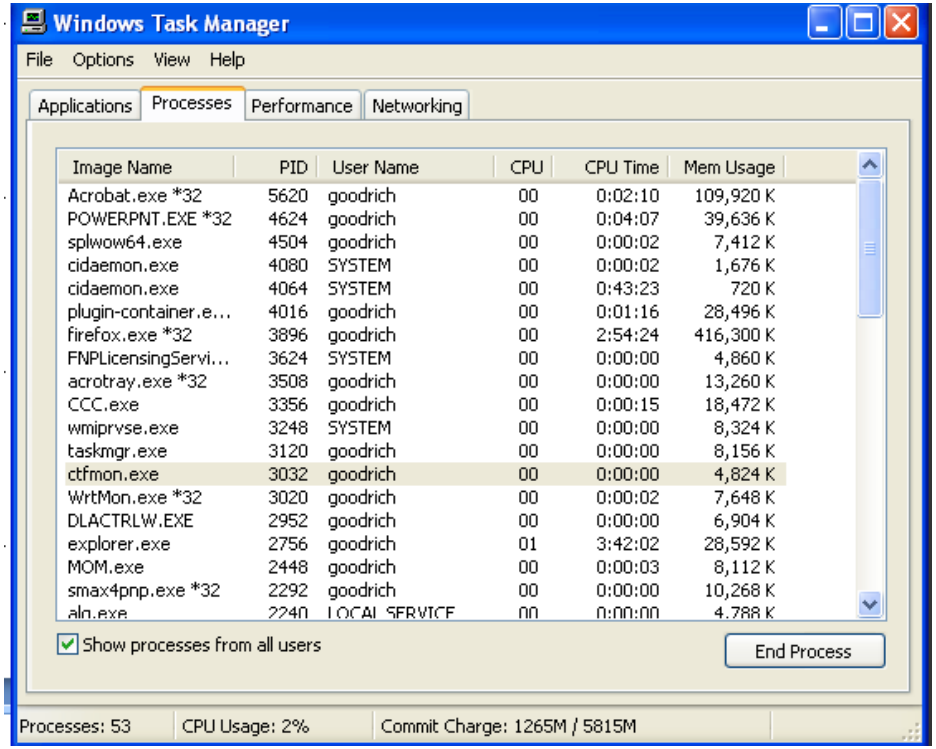
- New processes are created by **forking**
  - A new process is created by an existing process (e.g., a browser process – the ***child process*** - is created from a shell program – the ***parent process)***
  - Usually, child process inherits permissions of parent process
  - The parent-child relationships define a process tree
  - In Linux the root of the process tree is the **init** process
  - init becomes the parent of “orphaned” processes

Linux Process Tree

```
init─┬─NetworkManager─┬─dhclient
     |                └─{NetworkManager}
     ├─acpid
     ├─apache2─┬─apache2
     |         └─2*[apache2───26*[{apache2}]]
     ├─atd
     ├─avahi-daemon───avahi-daemon
     ├─bonobo-activati───{bonobo-activati}
     ├─console-kit-dae───63*[{console-kit-dae}]
     ├─cron
     ├─cupsd
     ├─2*[dbus-daemon]
     ├─2*[dbus-launch]
     ├─dd
     ├─devkit-disks-da───devkit-disks-da
     ├─devkit-power-da
     ├─gconfd-2
     ├─gdm-binary───gdm-simple-slav─┬─Xorg
     |                              └─gdm-session-wor───gnome-session─┬─bluetooth-apple
     |                                                                ├─gdu-notificatio
     |                                                                ├─gnome-panel
     |                                                                ├─gnome-power-man
     |                                                                ├─gnome-volume-co
     |                                                                ├─metacity
     |                                                                ├─nautilus
     |                                                                ├─nm-applet
     |                                                                ├─polkit-gnome-au
     |                                                                ├─2*[sh───sleep]
     |                                                                ├─ssh-agent
     |                                                                └─{gnome-session}
```

Process IDs

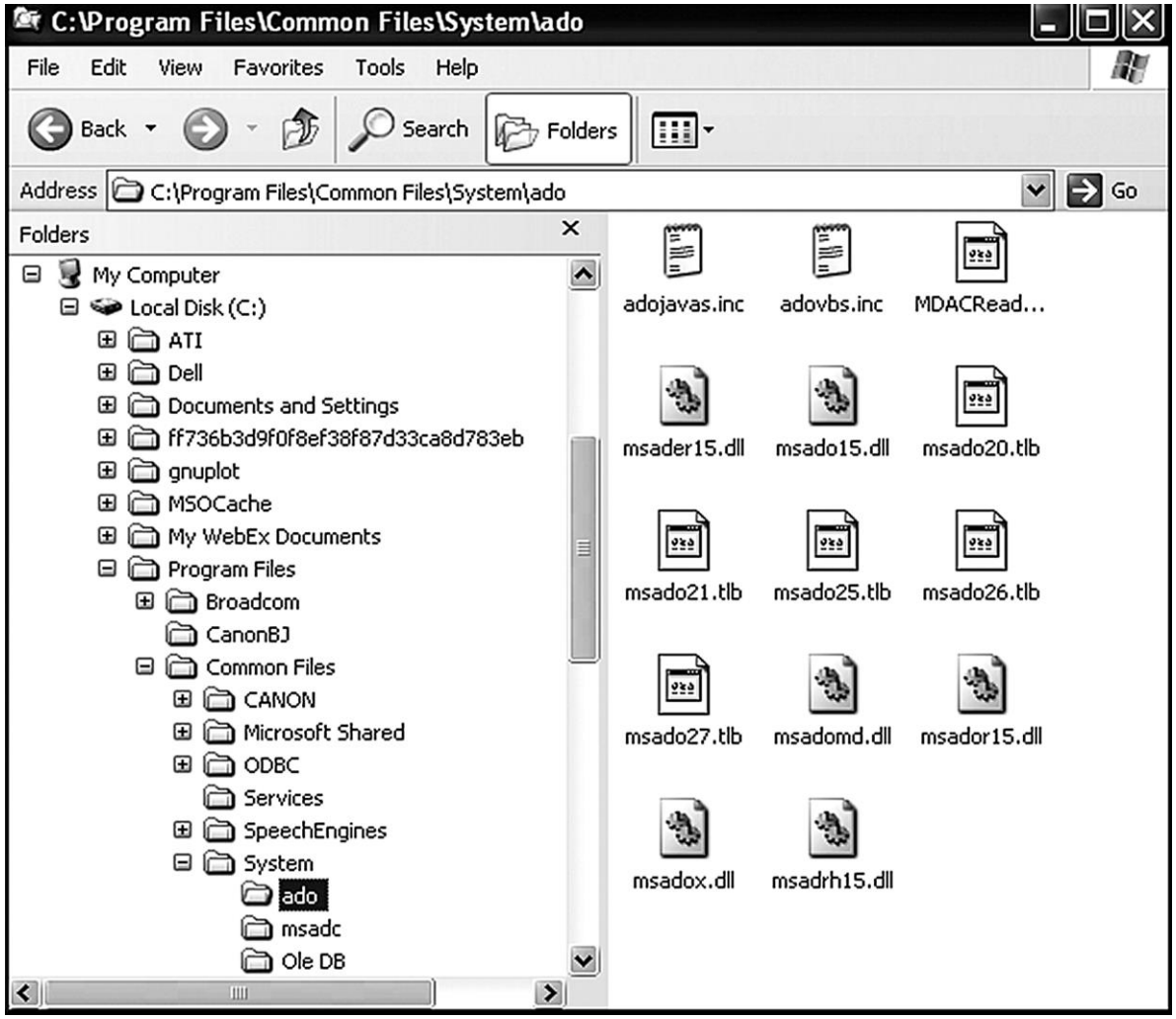
- Each process running on a given computer is identified by a unique nonnegative integer, called the **process ID (PID).**
- Given the PID for a process, we can then associate its CPU time, memory usage, user ID (UID), program name, etc.



File Systems

- A **filesystem** is an abstraction of how the external, nonvolatile memory of the computer is organized.
- Operating systems typically organize files hierarchically into **folders,** also called **directories.**
- Each folder may contain files and/or subfolders.
- Thus, a volume, or drive, consists of a collection of nested folders that form a **tree**.
- The topmost folder is the **root** of this tree and is also called the root folder.

File System Example



File Permissions

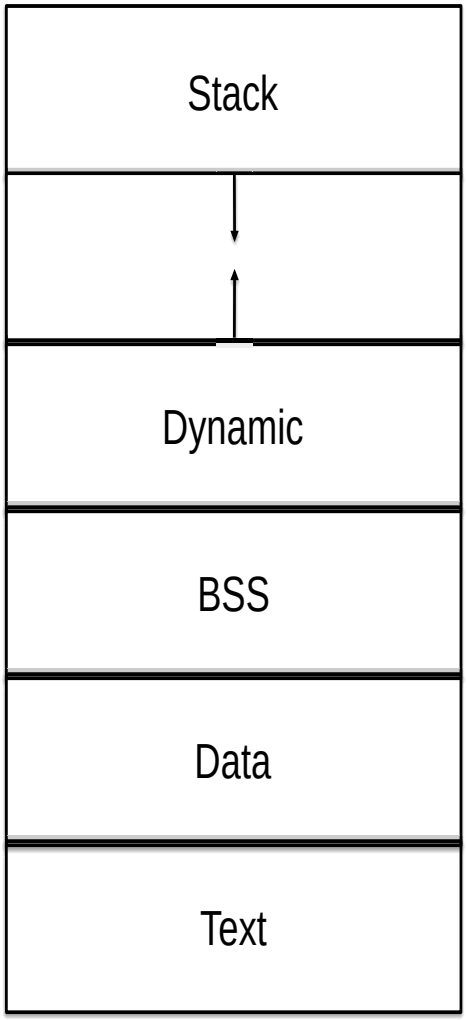
- File permissions are checked by the operating system to determine if a file is readable, writable, or executable by a user or group of users.
- In Unix-like OS's, a **file permission matrix** shows who is allowed to do what to the file.
  - **owner permissions** - show what the owner can do
  - **group permissions** - show what some group can do
  - **world permissions**, which give default access rights.

```
rodan:~/java % ls -l
total 24
-rwxrwxrwx   1 goodrich faculty     2496 Jul 27 08:43 Floats.class
-rw-r--r--   1 goodrich faculty     2723 Jul 12  2006 Floats.java
-rw-------   1 goodrich faculty      460 Feb 25  2007 Test.java
rodan:~/java %
```

Memory Management

- Region of memory allocated to a process is known as its **address space.**
- It contains both the code for the running program, its input data, and its working memory.
- For any running process, it is organized into different segments, which keep the different parts of the address space separate.
- As we will discuss, security concerns require that we never mix up these different segments.
- A process is not allowed to access the address space of another process

Memory Layout (used by most PCs)

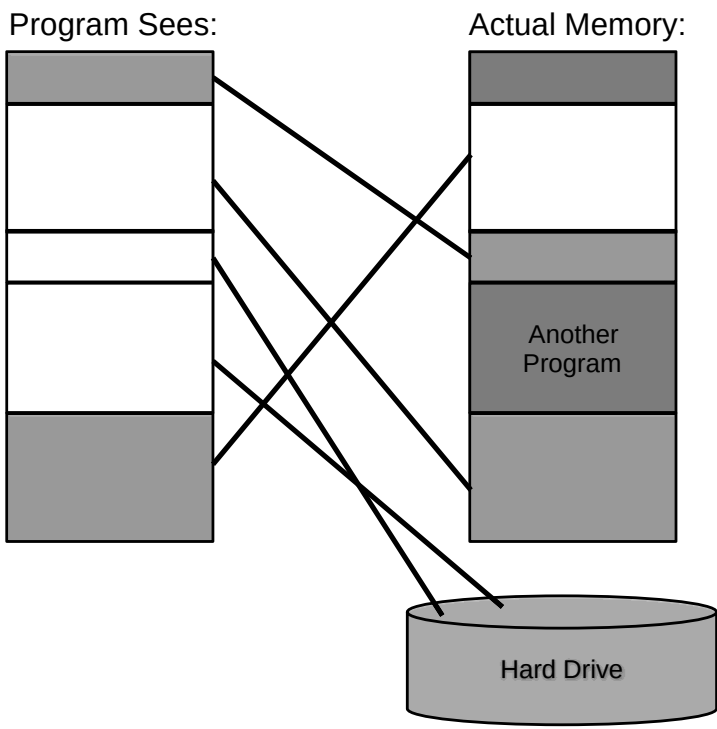


Memory Organization

- **Text.** This segment contains the actual (binary) machine code of the program.
- **Data.** This segment contains static program variables that have been initialized in the program code.
- **BSS.** This segment, which is named for an antiquated acronym for block started by symbol, contains static variables that are uninitialized.
- **Heap.** This segment, which is also known as the dynamic segment, stores data generated during the execution of a process.
- **Stack.** This segment houses a stack data structure that <u>grows downwards</u> and is used for keeping track of the call structure of subroutines (e.g., methods in Java and functions in C) and their arguments.

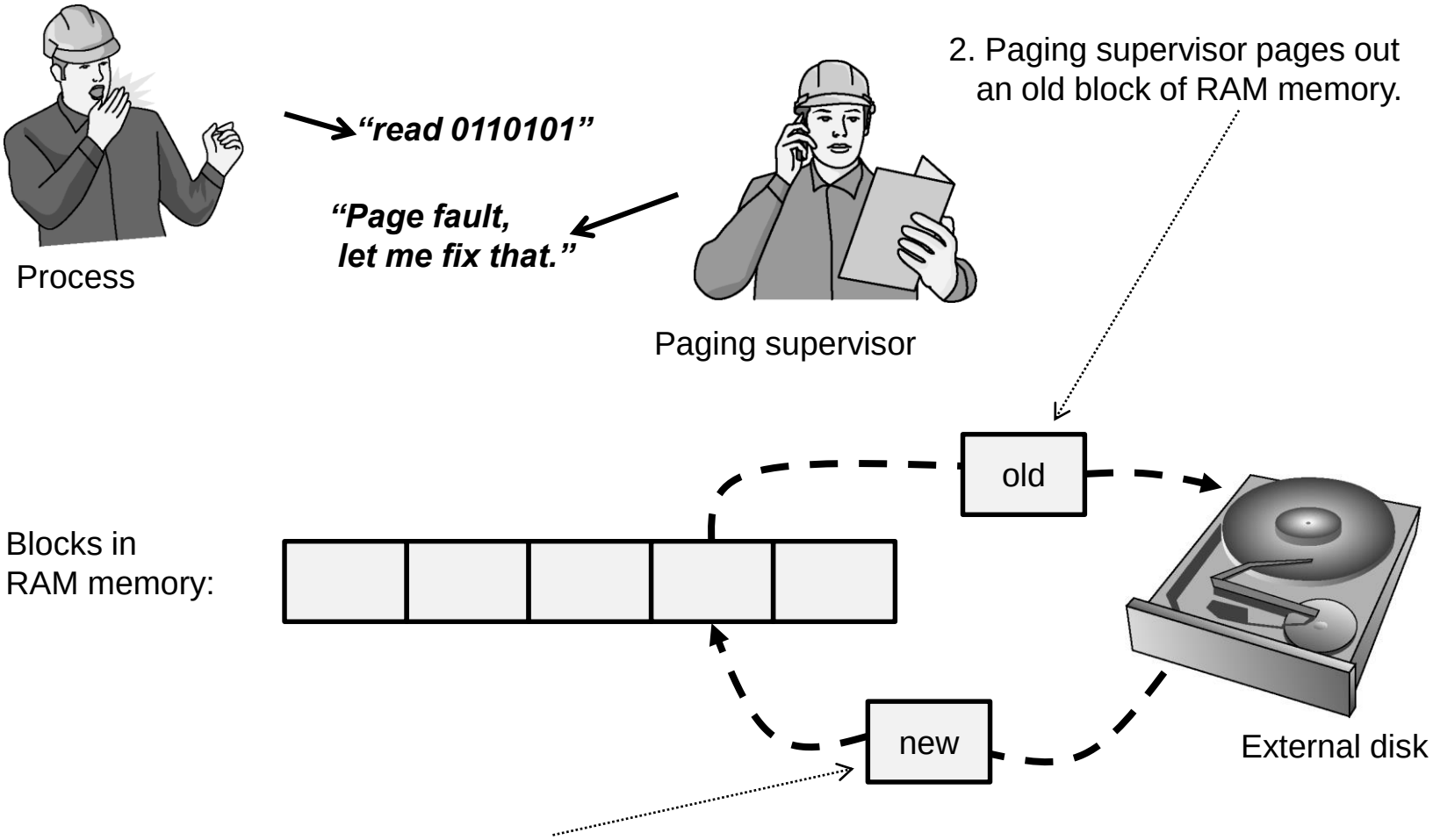
Virtual Memory

- There is generally not enough computer memory for the address spaces of all running processes.
- Nevertheless, the OS gives each running process the illusion that it has access to its complete (contiguous) address space.
- In reality, this view is **virtual**, in that the OS supports this view, but it is not really how the memory is organized.
- Instead, memory is divided into **pages**, and the OS keeps track of which ones are in memory and which ones are stored out to disk.
- Benefit: total size of address spaces of executing processes can be larger the actual physical memory of the computer



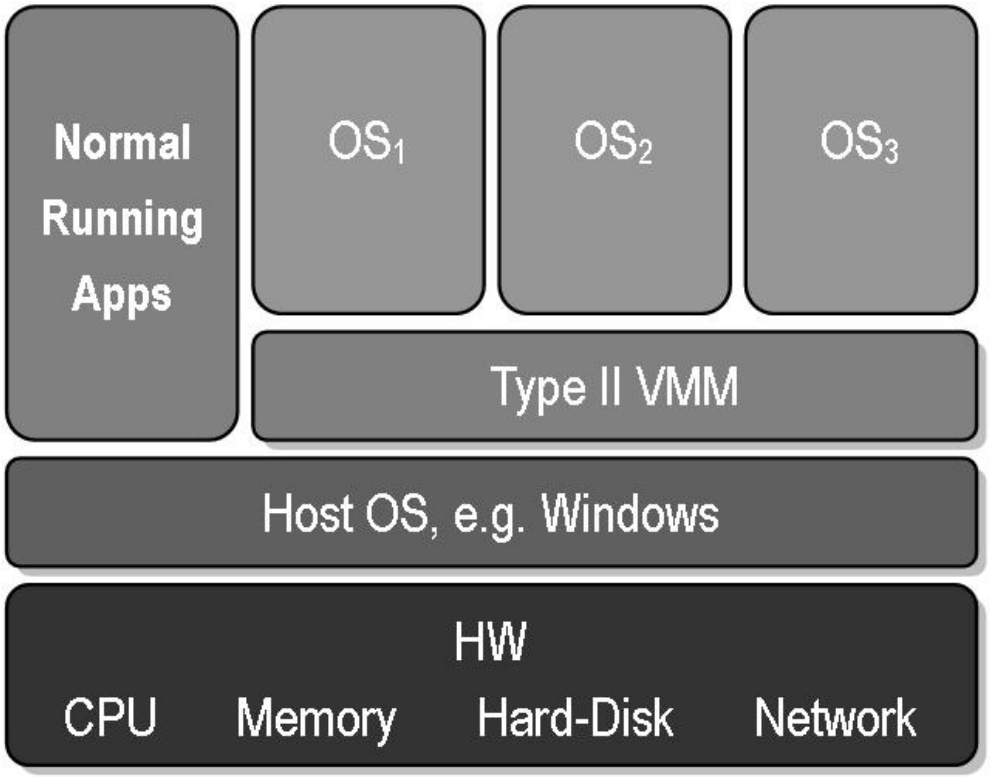
Page Faults

1. Process requests virtual address not in memory, causing a page fault.



Virtual Machines

- An operating system can run inside a virtual machine
- A hypervisor (virtual machine monitor – VMM) provides the environment for virtual machines
- **Benefits:**
  - **Hardware Efficiency**
  - **Portability**
  - **Security**
  - **Management**



Public domain image from http://commons.wikimedia.org/wiki/File:VMM-Type2.JPG

System Calls

- System calls are used by user applications to request a service from the kernel of the OS
  - They provide the interface between a user's process and the OS
- Processes are normally limited to their address space and are usually prevented to directly manipulate hardware devices
- System calls provide well-defined, safe implementations to allow processes access to the hardware devices

System Calls

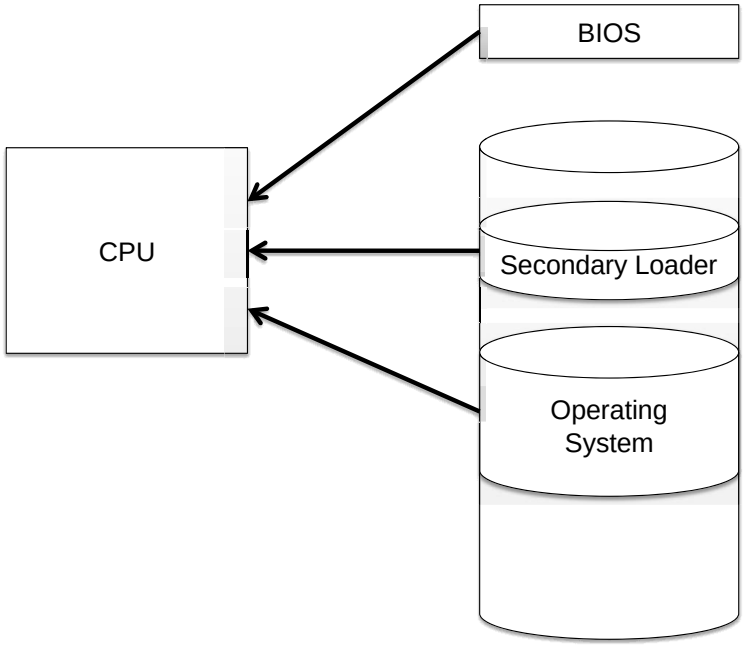
- A process makes a system call, which is usually executed via a ***software interrupt***
- An interrupt puts the CPU into some required privilege level (usually higher) and then passes control to the OS kernel, which determines whether the calling process should be granted the requested service
- If the service is granted:
  - the kernel executes a specific set of instructions over which the calling process has no direct control
  - the kernel returns the privilege level to that of the calling process
  - the kernel returns control to the calling process

System Calls – in practice

- The OS provides a library for system calls, usually an implementation of the C library (libc), such as ***glibc***
  - Library provides wrapper functions for the system calls
- Popular system calls on UNIX-like OS-es: ***open, read, write, close, wait, execve, fork, exit, kill***
  - Linux provides over 300 different system calls

The Boot Sequence

- The action of loading an operating system into memory from a powered-off state is known as **booting** or **bootstrapping.**
- When a computer is turned on, it first executes code stored in a firmware component known as the **BIOS (basic input/output system).**
- On modern systems, the BIOS loads into memory the **second-stage boot loader,** which handles loading the rest of the operating system into memory and then passes control of execution to the operating system.



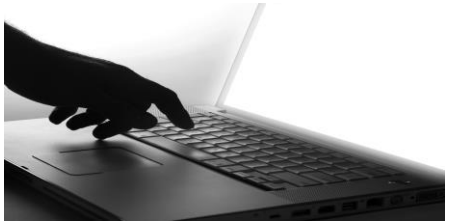
BIOS Passwords

- A malicious user could potentially seize execution of a computer at several points in the boot process.
- Boot device hierarchy
  - Could be exploited to bypass the OS authentication mechanisms
- To prevent an attacker from initiating the first stages of booting, many computers feature a **BIOS password** that does not allow a second-stage boot loader to be executed without proper authentication.

Hibernation

- Modern machines have the ability to go into a powered-off state known as **hibernation.**
- While going into hibernation, the OS stores the contents of machine's memory into a **hibernation file** (such as hiberfil.sys) on disk so the computer can be quickly restored later.
- But… without additional security precautions, hibernation exposes a machine to potentially invasive forensic investigation.

1. User closes a laptop computer, putting it into hibernation.



2. Attacker copies the hiberfil.sys file to discover any unencrypted passwords that were stored in memory when the computer was put into hibernation.

Event Logging

- “situational awareness”
  - Keeping track of what processes are running, what other machines have interacted with the system via the Internet, and if the operating system has experienced any unexpected or suspicious behavior can often leave important clues not only for troubleshooting ordinary problems, but also for determining the cause of a security breach.
- Operating systems have built-in logging mechanisms

Event Logging

- Windows: three sources of logs
  - “System”: can be written only by the OS
  - “Application”: can be written by user apps
  - “Security”: can be written by a special service *Local Security Authority Subsytem Service* (lsass.exe)
  - Example of log **events**: apps exiting unexpectedly, failed authentication attempts, network connections being made, etc.
- UNIX
  - Logs are simple text files stored in /var/log
  - Only a special **syslog** program can write the logs

Process Monitoring

- Operating systems have mechanisms to display the list of processes currently running
  - To identify how many resources each process is consuming
  - To check for suspicious processes
  - Usually allow for process termination
- Windows: **task manager**
- Linux: **ps**, **top**, **pstree**, **kill**

Process privileges

- To grant appropriate privileges to processes, the OS associates information about the user on whose behalf the process is being executed
- Each process has several user IDs:
  - Real user ID (uid) = the ID of the user associated with this process (the owner of this process, this is the user who has launched the process)
  - Effective user ID (euid) = the ID of the user with whose privileges this process is running
    - Usually, it is the same as the uid, but certain processes can run with the euid set to the ID of the program's owner (owner of the file)
    - The euid is used by the OS to decide whether to allow or deny actions for this process

Memory and Filesystem Security

- The contents of a computer are encapsulated in its memory and filesystem.
- Thus, protection of a computer's content has to start with the protection of its memory and its filesystem.

Attacks on Virtual Memory

- Virtual memory contains the memory pages that have been written to the hard disk
  - Windows: **page file**, C:\pagefile.sys
  - Linux: **swap** partition
  - The OS enforces rules preventing users from viewing the contents of virtual memory files while the OS is running; those files are usually **deleted when the machine is shut down**.
- Attacks
  - Shut down the machine improperly (e.g., sudden power off)
  - Boot via LiveCD to read these files

Password-based Security

- One approach to guessing passwords from the password file is to conduct a **dictionary attack**, where each word in a dictionary is hashed and the resulting value is compared with the hashed passwords stored in the password file.
- A dictionary of 500,000 “words” is often enough to discover most passwords.

Password Salt

- One way to make pre-computed dictionary attacks more difficult to execute is to use **salt**.
- Associate a random number with each userid.
- Rather than comparing the hash of an entered password with a stored hash of a password, the system compares the hash of an entered password and the salt for the associated userid with a stored hash of the password and salt.

How Password Salt Works

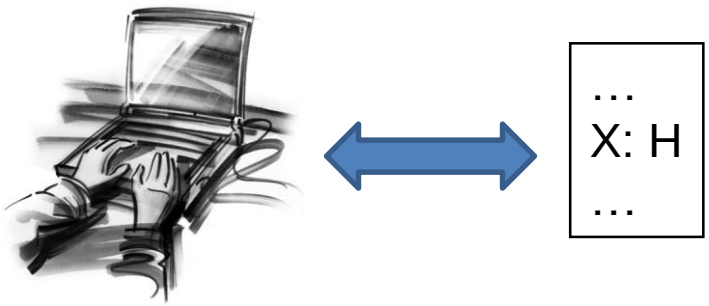
Without salt:

1. User types userid, X, and password, P.

2. System looks up H, the stored hash of X's password.

3. System tests whether h(P) = H.

Password file:



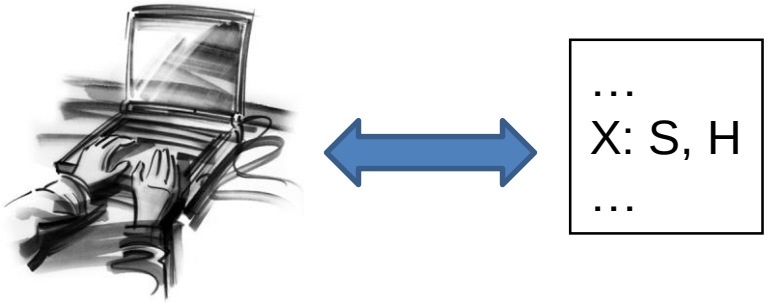
With salt:

1. User types userid, X, and password, P.

2. System looks up S and H, where S is the random salt for userid X and H is stored hash of S and X's password.

3. System tests whether h(S||P) = H.

Password file:



How Salt Increases Search Space Size

- Assuming that an attacker cannot find the salt associated with a userid he is trying to compromise, then the search space for a dictionary attack on a salted password is of size

  $$2^B * D,$$

  where B is the number of bits of the random salt and D is the size of the list of words for the dictionary attack.
- For example, if a system uses a 32-bit salt for each userid and its users pick passwords in a 500,000 word dictionary, then the search space for attacking salted passwords would be

  $$2^{32} * 500{,}000 = 2{,}147{,}483{,}648{,}000{,}000,$$

  which is over 2 quadrillion.
- Also, even if an attacker can find a salt password for a userid, he only learns one password.

Password Authentication in Windows

- Password hashes are stored in a file called *Security Account Manager (SAM)* file, which is not accessible to regular users
  - Passwords are hashed using a combination of MD4 hash and a custom-brewed LAN Manager (LM) hash

Password Authentication in Linux

- Passwords are stored in /etc/passwd file, usually in conjunction with /etc/shadow file
- Format of shadow file

vivek:1fnfffc$pGteyHdicpGOfffXX4ow#5:13064:0:180:7:::

  - User name
  - Hash value: $algo$salt$hash
    - Algorithm can be 1 (md5), 2 (blowfish), 5 (sha256), 6 (sha512)
  - Last password change (number of days since Jan 1, 1970)
  - Minimum number of days before password may be changed
  - Maximum number of days before which password must be changed (99999 is indefinite).
  - Number of days before password expiration when user will be warned
  - Other fields: Number of days after password expiration when account will be disabled; Number of days since January 1, 1970 that an account has been disabled; Field is reserved for future use.

Filesystem Security

Filesystem Security – general principles

- Files and folders are managed by the operating system
- A principal = a user or a group of users
  - Example, in UNIX: user ***owner*** (owner of the file), group ***group*** (the default group associated with the file), group ***all*** (all the users), group ***other*** (all users except the owner of the file)
- Permission – a specific action on a file of folder
  - Read, write, execute
- Access control entry (ACE)
  - A triplet (principal, type, permission)
  - Type is either allow/deny
  - Allow/deny a certain type of access to a file/folder by user/group
- Access control list (ACL)
  - Collection of ACEs for a file/folder

Access Control Policies

- Discretionary Access Control (DAC)
  - Users can protect what they own
  - The owner may grant access to others
  - The owner may define the type of access (read/write/execute) given to others
  - DAC is the standard model used in operating systems
- Mandatory Access Control (MAC)
  - Example: Security-Enhanced Linux (SELinux) developed by the US NSA
  - Every allowable action on a machine must be defined
  - Multiple levels of security for users and documents
  - Users are not given the power to decide security attributes of their own files; this is delegated to a central policy administrator
  - This is an alternative model not covered in the course

Closed vs. Open Policy

Closed policy

- Also called “default secure”
- Give Tom read access to “foo”
- Give Bob r/w access to “bar”
- Tom: I would like to read “foo”
  - Access allowed
- Tom: I would like to read “bar”
  - Access denied

Open Policy

- Deny Tom read access to “foo”
- Deny Bob r/w access to “bar”
- Tom: I would like to read “foo”
  - Access denied
- Tom: I would like to read “bar”
  - Access allowed

Closed Policy with Negative Authorizations and Deny Priority

- Give Tom r/w access to “bar”
- Deny Tom write access to “bar”
- Tom: I would like to read “bar”
  - Access allowed
- Tom: I would like to write “bar”
  - Access denied
- Policy is used by Windows to manage access control to the file system

Users, Groups, Processes, Files

- Each user has a unique UID
- Each group has a unique GID
- Each process has a unique PID
- Users belong to multiple groups GID
- Objects whose access is controlled
  - Files
  - Directories

Organization of Objects

- Files are arranged in a hierarchy
- Files exist in directories
- Directories are one type of files

- In UNIX, access rights on directories are not inherited

Basic Permissions Bits on Files

- Read controls reading the content of a file
  - i.e., the read system call
- Write controls changing the content of a file
  - i.e., the write system call
- Execute controls loading the file in memory and execute
  - i.e., the execve system call
- Many operations can be performed only by the owner of the file

Where are Permission Bits Kept?

- Each file/directory has associated an **inode**.
- The file type, permissions, owner UID and owner GID are saved on disk in the inode of a file or directory

Execution of a File

- Binary file vs. script file
- Having execute but not read, can one run a binary file?
- Having execute but not read, can one run a script file?
- Having read but not execute, can one run a script file?

File Permissions

- File permissions are often displayed in a compact 10-character notation:

  drwxr-xr-x

- To see permissions, use **ls –l**

```
rodan:~/java % ls -l
total 24
-rwxrwxrwx   1 goodrich faculty     2496 Jul 27 08:43 Floats.class
-rw-r--r--   1 goodrich faculty     2723 Jul 12  2006 Floats.java
-rw-------   1 goodrich faculty      460 Feb 25  2007 Test.java
rodan:~/java %
```

- First character: indicates the type of file
  - directory (d), regular file (-), etc.
- Next three: owner permission
- Next three: group permission
- Next three: others permission

File Permissions Examples

-rw-r--r--	read/write for owner, read-only for everyone else
-rw-r-----	read/write for owner, read-only for group, forbidden to others
-rwx------	read/write/execute for owner, forbidden to everyone else
-r--r--r--	read-only to everyone, including owner
-rwxrwxrwx	read/write/execute to everyone

Permission Bits on Directories

- Read bit allows listing of file names in a directory, but not the listing of file properties like size and permissions
- The execute bit allows traversing (entering) the directory and getting properties of files in the directory
  - chdir to a directory requires execute privilege
- Write bit controls creating/deleting files in the directory
  - Deleting/creating a file under a directory requires no permission on the file
- Accessing a file identified by a path name requires execution to all directories along the path
  - Example: Bob is currently accessing directory /home/alice (for which he has execute permission) and issues the Unix command

    cat administration/memos/raises.txt

Permissions examples (on directories)

drwxr-xr-x	all can enter and list the directory, only owner can add/delete files
drwxrwx---	full access to owner and group, forbidden to others
drwx--x---	full access to owner, group can access known filenames in directory, forbidden to others
drwxrwxrwx	full access to everyone

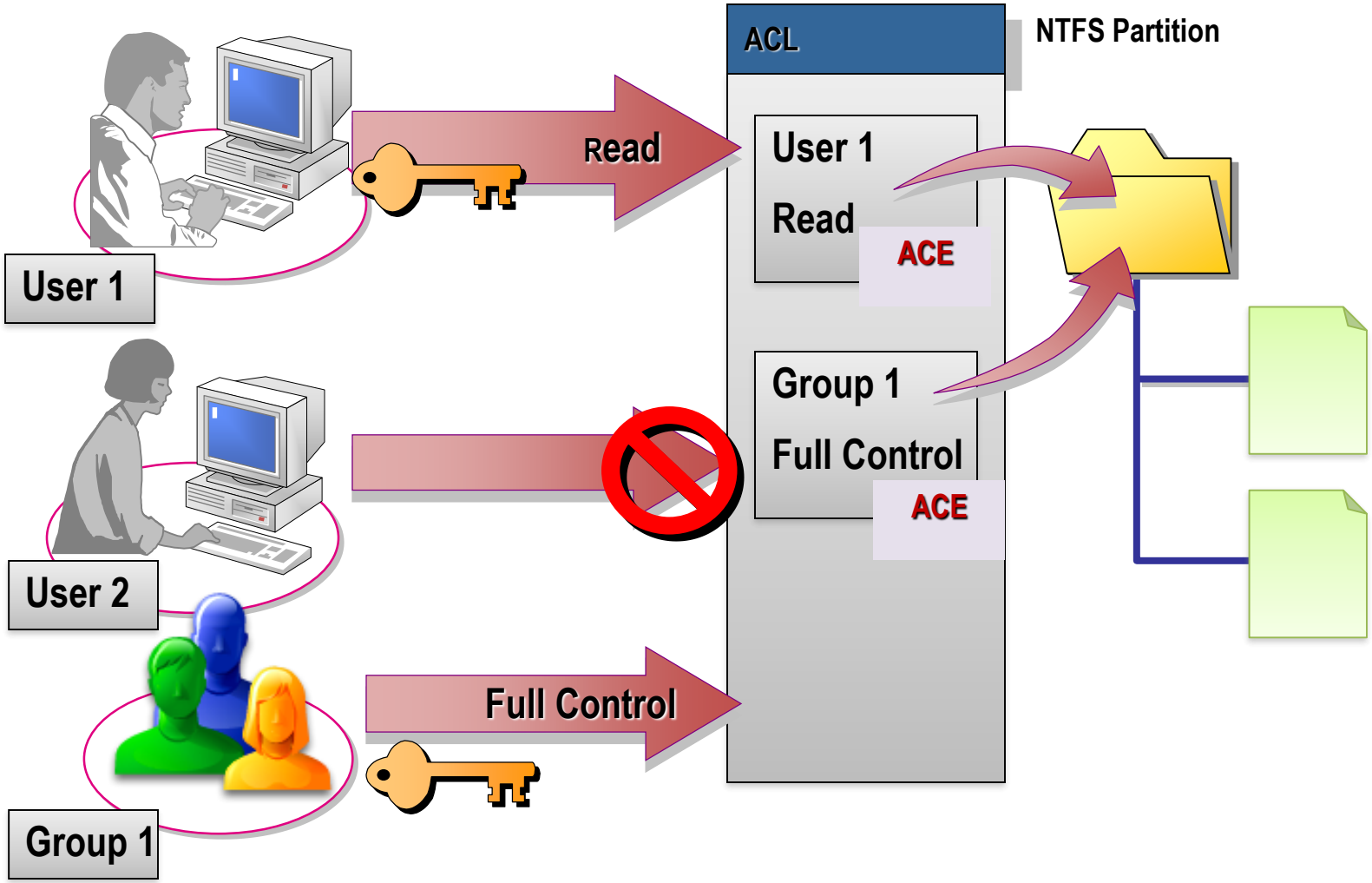
The Three Sets of Permission Bits

- Order:
  - if the user is the owner of a file
    - then the r/w/x bits for **owner** apply
  - otherwise, if the user belongs to the group the file belongs to
    - then the r/w/x bits for **group** apply
  - otherwise,
    - the r/w/x bits for **others** apply

Windows Permissions

- ACL model that allows users to create sets of rules for each user or group
- The rules either allow or deny various permissions for the principal
- If there is no applicable rule, access is denied by default

NTFS Permissions



Basic NTFS Permissions

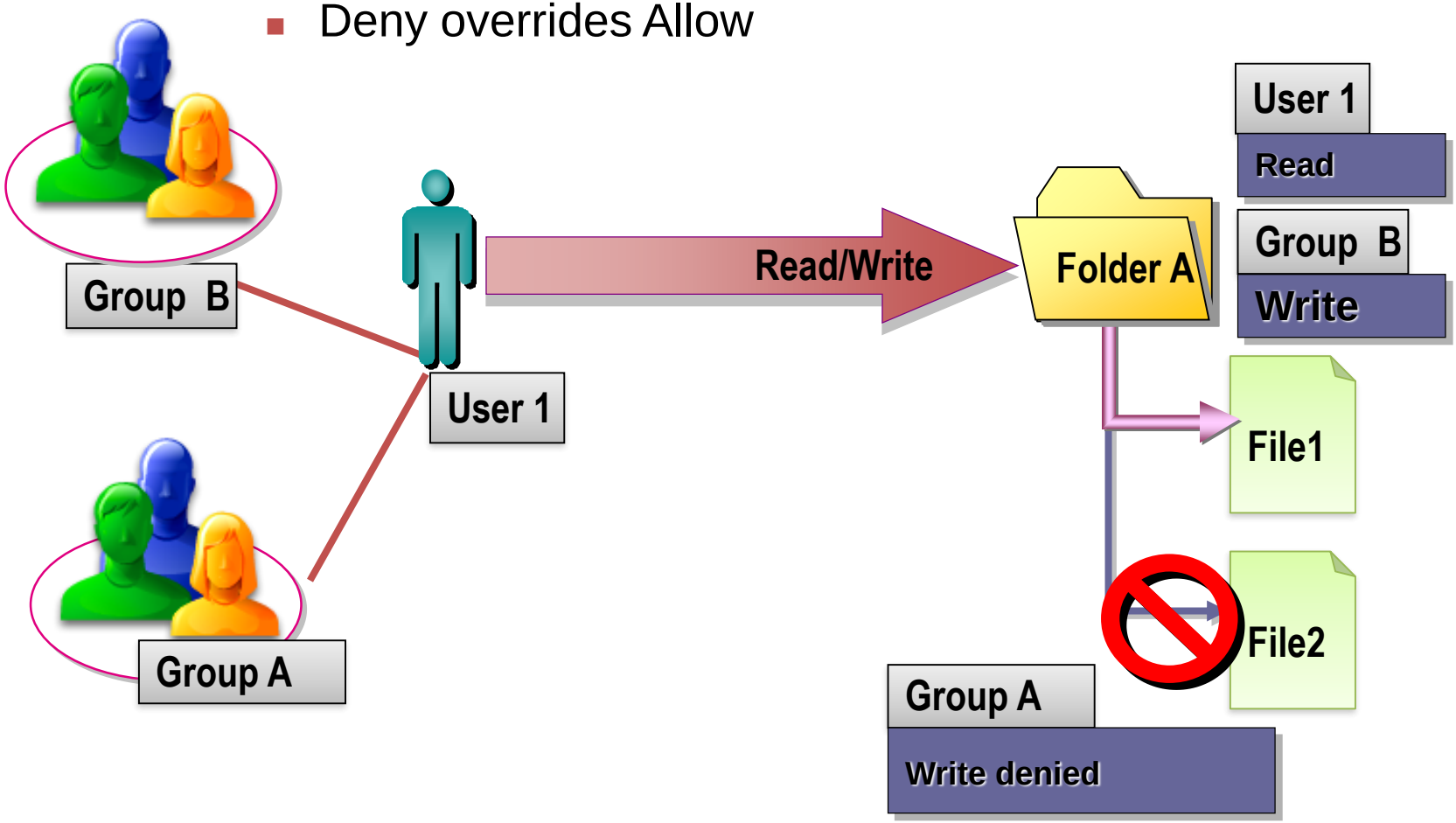
NTFS Permission	Folders	Files
Read	Open files and subfolders	Open files
List Folder Contents	List contents of folder, traverse folder to open subfolders	Not applicable
Read and Execute	Not applicable	Open files, execute programs
Write	Create subfolders and add files	Modify files
Modify	All the above + delete	All the above
Full Control	All the above + change permissions and take ownership, delete subfolders	All the above + change permissions and take ownership

Advanced Permissions

- To finely tune permissions, Windows uses advanced permissions, which the Basic Permissions are composed of
- For example: the basic read permission encompasses several advanced permissions:
  - Read data
  - Read attributes
  - Read extended attributes
  - Read permissions

Multiple NTFS permissions

- NTFS permissions are cumulative
- File permissions override folder permissions
- Deny overrides Allow

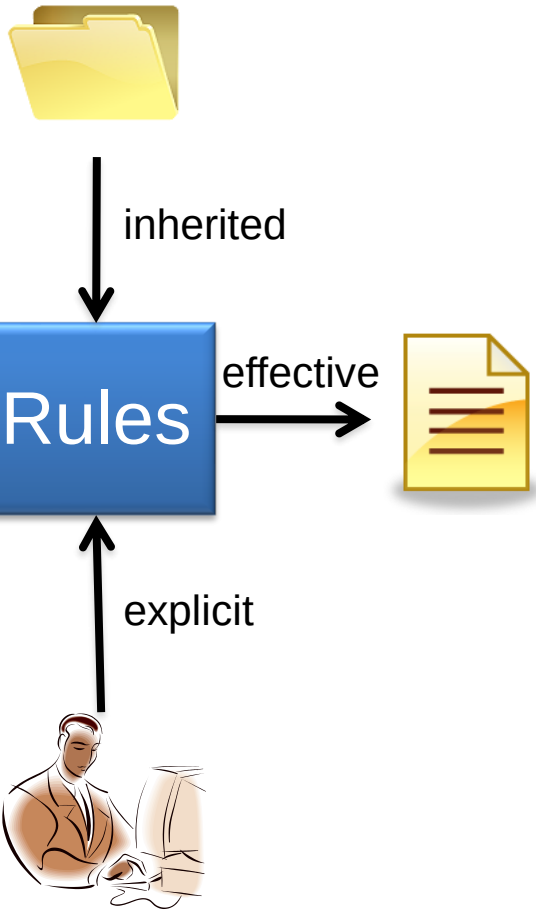


NTFS File Permissions

- **Explicit**: set by the *owner* for each user/group.
- **Inherited**: dynamically inherited from the explicit permissions of ancestor folders.
- **Effective**: obtained by combining the explicit and inherited permission.

Determining effective permissions:

- By default, a user/group has no privileges.
- Explicit permissions override conflicting inherited permissions.
- Denied permissions override conflicting allowed permissions.



NTFS File Permissions

- Example: an administrator can deny access to a folder for a user, but allow the user access to a file within that folder
  - This is not possible in Linux

- The hierarchy of folders, along with finely granulated control of Windows permissions, provides admins with substantial flexibility
  - However, it may create security holes due to its complexity

Special Permission Bits

- Three other permission bits exist
  - Sticky bit
  - Set-user-ID (“suid” or “setuid”) bit
  - Set-group-ID (“sgid” or “setgid”) bit

Sticky Bit

- For a directory:
  - When sticky bit is set, only the file owner, the directory owner or the superuser can rename or delete files within the directory
  - When sticky bit is not set, any user with write and execute permissions for the directory can rename and delete files within the directory, regardless of owner
  - Typically, the sticky bit is set on the /tmp directory to prevent ordinary users from deleting or renaming other users' files
- For files, its use is obsolete

Sticky Bit

- In 10-character display, replaces 10th character:
  - x becomes t (files which are executable)
  - - becomes T (files which are not executable)

drwxrwxrwt

- sticky bit set, full access for everyone

drwxrwx--T

- sticky bit set, full access by user/group

drwxr--r-T

- sticky, full owner access, others can read *(useless)*

Users vs. Subjects

- Above discussions are about which users (accounts) can access a file
- Actions are performs by subjects (processes)
- When a process accesses a file, the OS needs to know on which user's behalf is the process acting

Example

- Consider the **passwd** program
- Needs to update a system-wide password file, which ordinary users should not be able to modify, but only root can modify
- Needs to be run by ordinary users

How to support such cases?

User ID vs. Effective User ID

- Super-user ID is 0 and has no restrictions, any user with ID 0 would have the super-user rights
- There are several user IDs associated with a process:
  - User ID (uid) = the ID of the user associated with this process (this is the user who executed the program and launched the process)
  - Effective user ID (euid) = the ID of the user with whose privileges this process is running
    - Usually, it is the same as the uid, but certain processes run with the euid set to the ID of the program's owner (owner of the file)
    - The euid is used by the OS to decide whether to allow or deny actions for this process

User ID vs. Effective User ID

- Each process also has several group IDs:
  - real group ID (gid)
  - effective group ID (egid)

setuid bit

- ***setuid*** bit:
  - An additional bit in the file permission matrix
  - If this bit is set, then a program runs with the effective user ID of its owner, rather than the ID of the user that executed the program
  - Example: *passwd* is owned by ***root***, has execute bit set for the ***others*** class, and has the ***setuid*** bit set
    - When a user runs passwd, the program runs with the permissions of the root user, allowing it to alter the /etc/passwd file (which can only be written by the root user)
- ***setgid*** bit functions similarly, but for groups

setuid bit

- ***setuid*** bit
  - On executable files, causes the program to run as file owner regardless of who runs it
  - Ignored for everything else
  - In 10-character display, replaces the 4th character:
    - x becomes s (for files which are executable)
    - - becomes S (for files which are not executable)

-rwsr-xr-x

- setuid, executable by all

-rwxr-xr-x

- executable by all, but not setuid

-rwSr--r--

- setuid, but not executable - not useful

Example of a program with setuid bit set

```c
#include <stdio.h>
#include <sys/types.h>
#include <unistd.h>
#include <stdlib.h>

static uid_t euid, uid;

int main(int argc, char *argv[])
{
  FILE *file;

  /*store real and effective user IDs*/
  uid = getuid();
  euid = geteuid();

  /*drop priviliges*/
  seteuid(uid);

  /*do something useful*/

  /*raise privileges*/
  seteuid(euid);

  /*Open the file*/
  file = fopen("/home/admin/log", "a");

  /*drop priviledges again*/
  seteuid(uid);

  /*write to the file*/
  fprintf(file, "Someone used this program.\n");


  /*close the file stream and return*/
  fclose(file);

  return 0;

}
```

More Examples

- I posted a file on the course website with more examples of uid and euid

Root

- “root” account is a super-user account, like Administrator on Windows
- Multiple roots possible
- File permissions do not restrict root
- This is *dangerous*, but necessary, and OK with good practices

Becoming Root

- su
  - Changes home directory, PATH, and shell to that of root, but doesn't touch most of environment and doesn't run login scripts
- su -
  - Logs in as root just as if root had done so normally
- sudo <command>
  - Run just one command as root
- su [-] <user>
  - Become another non-root user
  - Root does not require to enter password

Read more at:

https://linuxacademy.com/blog/linux/linux-commands-for-beginners-sudo/

Changing Permissions

- Permissions are changed with chmod or through a GUI
- Only the file owner or root can change permissions

- If a user owns a file, the user can use chgrp to set its group to any group of which the user is a member

- root can change file ownership with chown (and can optionally change group in the same command)

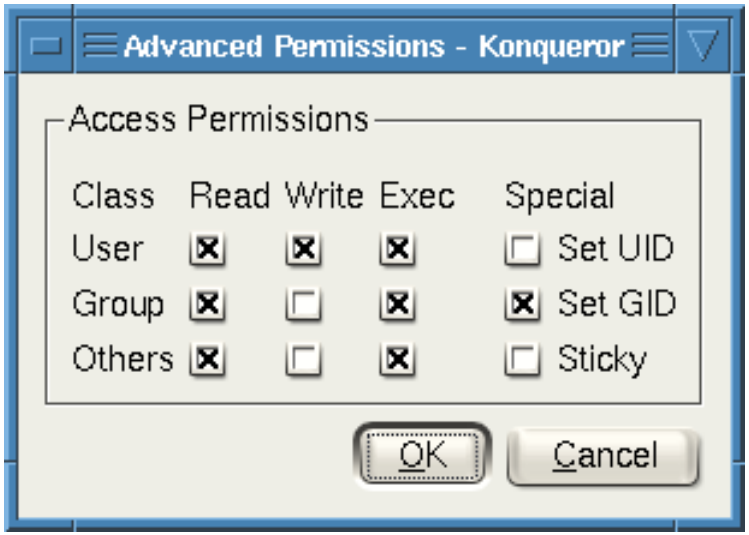
- chown, chmod, and chgrp can take the –R option to recursively apply the command through subdirectories

Examples of Changing Permissions

chown -R root dir1	Changes ownership of dir1 and everything within it to root
chmod g+w,o-rwx file1 file2	Adds group write permission to file1 and file2, denying all access to others
chmod -R g=rwX dir1	Adds group read/write permission to dir1 and everything within it, and group execute permission on files or directories where someone has execute permission
chgrp testgrp file1	Sets file1's group to testgrp, if the user is a member of that group
chmod u+s file1	Sets the setuid bit on file1. (Doesn't change execute bit.)

Working Graphically with Special Bits

- Special permission bits can also be displayed and changed through a GUI
- In Konqueror's Permissions window, click Advanced Permissions:
- Changes can be made here (more about changes later)



Octal Notation

- Previous syntax is nice for simple cases, but bad for complex changes
  - Alternative is octal notation, i.e., three or four digits from 0 to 7
- Digits from left (most significant) to right(least significant):

  [special bits][user bits][group bits][other bits]
- Special bit digit =

  (4 if setuid) + (2 if setgid) + (1 if sticky)
- All other digits =

  (4 if readable) + (2 if writable) + (1 if executable)

Octal Notation Examples

644 or 0644	read/write for owner, read-only for everyone else
775 or 0775	read/write/execute for owner and group, read/execute for others
640 or 0640	read/write for owner, read-only for group, forbidden to others
2775	same as 775, plus setgid (useful for directories)
777 or 0777	read/write/execute to everyone *(dangerous!)*
1777	same as 777, plus sticky bit

File Descriptors

- Processes need a shortcut to access files, (other than always going to the filesystem and specifying a path to the files)
- ***File descriptors*** allow processes to efficiently read and write files stored on disk
  - A file descriptor is essentially an index stored in the ***file descriptor table***

File Descriptors – how do they work?

- When a process needs to access a file, it calls the *open* system call
- The OS kernel checks that the calling process has permission to access the file in the manner requested
- Kernel creates a new entry in the file descriptor table, which maps to the file's location on the disk
- The file descriptor is returned to the process
- When a call to read/write is issued, the kernel looks the file descriptor in the table and checks if the file descriptor has the appropriate permissions set; if not, the read/write fails and the process typically halts
- Otherwise, kernel performs the read/write
- The process should finally issue the close system call to remove the open file descriptor

File Descriptors – what can go wrong?

- When a process creates a child process (using fork), the child inherits copies of all the file descriptors that are open in the parent.
- The OS checks whether a process had read/write permissions for a file only at the moment of creating a file descriptor entry
- When a process issues a read/write to a file, the OS only confirms that the action is allowed according to the permissions the file descriptor was created with

File Descriptors – File descriptor leak

- A program with high privileges:
  - Opens a file descriptor to a protected file
  - Creates a process that has lower privileges before closing the file descriptor to the protected file

File descriptor leak example

```
#include <stdio.h>
#include <unistd.h>


int main(int argc, char *argv[])
{
  FILE *passwords;

  /*open the password file for reading*/
  passwords = fopen("/home/admin/passwords", "r");

  /*read the passwords and do something useful*/
  fclose(passwords)
  /*fork and execute Joe's shell without closing the file*/
  execl("/home/joe/shell", "shell", NULL);
}
```

Links in the Linux Filesystem

- In addition to regular files, there are links
- Hard link
  - It is an entry in a directory that associates a name with a file
  - The same file can have hard links from multiple directories, each with its own filename, but all sharing owner, group, and permissions
  - Changes to the file content are reflected in all the hard links pointing to the file
  - File is only deleted when no more hard links to it
- Symbolic link (symlink) or soft link
  - A special type of file that simply contains a reference to another file or directory
  - Stores path to target, which is traversed for each access
  - The same file or directory can have multiple symlinks to it
  - Removal of symlink does not affect target
  - Removal of target invalidates (but does not remove) symlinks to it
  - Analogue of Windows shortcut (with an important difference though)

Links in the Linux Filesystem

- How to create them?
- Hard links:
  - ln source target
- Soft links:
  - ln –s source target

Symlinks – what can go wrong?

- Imagine a program that opens and reads a file specified by the user; however, the program should prohibit the read of a particular file, say /home/admin/passwords

- Unsafe version of program simply checks if the user provided file name is different than "/home/admin/passwords"
- Attacker can trick the program by creating a symlink to that file and simply providing the path to the symlink to the program

- The program should either check if the provided filename refers to a symlink, or check the actual filename being open by using a ***stat*** system call

What to read?

- For OS security: Chapters 3.1, 3.2, 3.3 from the Textbook
- For filesystem security: Chapter 3.3 from the Textbook